

Laboratorio S5

October 4, 2026

1 Visión por Computadora

1.1 Clase 5 — Introducción al aprendizaje profundo para visión

1.1.1 Unidad 2 · Redes convolucionales y modelos avanzados

Laboratorio práctico (Jupyter). Durante toda la Unidad 1 *tú* decidiste qué mirar en una imagen: diseñaste filtros (Sobel, Canny), elegiste descriptores (SIFT, HOG) y fijaste las reglas para segmentar. Hoy empieza el gran giro del curso: **la red aprende sola qué mirar, a partir de ejemplos.**

Vas a entrenar tu **primer modelo en PyTorch** sobre **MNIST**, un conjunto de 70 000 dígitos escritos a mano. Recorrerás la maquinaria completa del aprendizaje profundo: la imagen como **tensor**, el **modelo** como una función con **parámetros** ajustables (como las cantidades de una receta), la **pérdida** que mide el error, el **gradiente** que indica cómo corregir y el **loop de entrenamiento** que repite todo eso miles de veces.

El modelo de hoy es un **perceptrón multicapa (MLP)**. Funciona bien con dígitos, pero al final descubrirás su límite —**aplanar la imagen destruye la estructura espacial**— y ese límite es exactamente lo que la **convolución** resuelve en la Clase 6.

Cómo trabajar en este notebook

1. Ejecuta las celdas **en orden**, de arriba hacia abajo, con el botón  o con **Shift + Enter**.
2. Antes de correr cada celda, intenta **predecir** qué va a pasar.
3. Cada línea de código lleva un comentario (lo que va después de #) que explica qué hace. Léelos: son parte de la clase.
4. La sección **TU TURNO**, al final, es para que experimentes tú.

2 Se esta corriendo en Jupyter si es necesario instalar PyTorch, torchvision, OpenCV, NumPy y Matplotlib.

Opcional, para ir más rápido: menú *Entorno de ejecución* → *Cambiar tipo de entorno de ejecución* → *GPU (T4)*. Sin GPU también funciona; el entrenamiento solo tarda un poco más.

[]: ---

2.1 Parte 0 — Preparar el entorno

Traemos las librerías que usaremos. La novedad de hoy es **PyTorch** (`torch`), la librería de aprendizaje profundo, y **torchvision**, su compañera para visión, que trae datasets como MNIST.

```
[2]: # -----  
# Cada 'import' trae una librería (una caja de herramientas ya hecha).  
# La palabra "as" le pone un apodo corto para escribir menos.  
# -----  
import numpy as np                # NumPy: tablas de números (como en la Unidad 1)  
import matplotlib.pyplot as plt    # Matplotlib: para dibujar imágenes y gráficos  
import cv2                        # OpenCV: lo usaremos al final para preparar tu foto  
import torch                      # PyTorch: la librería de aprendizaje profundo  
import torch.nn as nn            # nn = "neural networks": capas, pérdidas, modelos  
from torch.utils.data import DataLoader # DataLoader: sirve los datos al modelo en lotes  
from torchvision import datasets, transforms # datasets (MNIST) y transformaciones de imágenes  
  
# Hace que los gráficos aparezcan DENTRO del notebook, debajo de cada celda.  
%matplotlib inline  
  
# Fijamos la "semilla" del azar: así todos obtienen resultados muy parecidos.  
torch.manual_seed(42)             # semilla para PyTorch  
np.random.seed(42)               # semilla para NumPy  
  
# Si Colab nos dio una GPU la usamos ("cuda"); si no, trabajamos con el procesador ("cpu").  
device = "cuda" if torch.cuda.is_available() else "cpu"  
  
print("PyTorch versión:", torch.__version__) # confirma que PyTorch está disponible  
print("Vamos a entrenar en:", device)        # "cuda" = GPU, "cpu" = procesador
```

PyTorch versión: 2.13.0+cpu

Vamos a entrenar en: cpu

2.2 Parte 1 — La imagen como tensor

En la Unidad 1 una imagen era un `array` de NumPy: una rejilla de números. En PyTorch esa misma rejilla se llama **tensor**. No es algo misterioso: es el mismo dato, con dos superpoderes que

el aprendizaje profundo necesita.

1. Puede vivir en la **GPU**, que hace miles de cuentas a la vez.
2. Puede **recordar cómo se calculó**, y gracias a eso la red sabrá cómo corregirse (lo veremos en la Parte 7).

Empezamos, como siempre, **de menos a más**: una imagen diminuta de 3×3 escrita a mano.

```
[3]: # Una "imagen" de 3x3 píxeles escrita a mano con NumPy (valores de gris entre 0
    ↪y 255).
mini_np = np.array([[ 0, 128, 255],      # fila de arriba: negro, gris, blanco
                    [ 64, 128, 192],     # fila del medio
                    [255, 128,  0]],      # fila de abajo: blanco, gris, negro
                    dtype=np.float32)     # float32 = números con decimales (lo
    ↪que usa PyTorch)

# torch.from_numpy convierte el array de NumPy en un tensor de PyTorch (mismos
    ↪números).
mini_t = torch.from_numpy(mini_np)

print("Como array de NumPy:\n", mini_np)  # la rejilla que ya conoces
print("\nComo tensor de PyTorch:\n", mini_t) # la MISMA rejilla, ahora como
    ↪tensor
print("\nForma (shape):", mini_t.shape)    # torch.Size([3, 3]) -> 3 filas y
    ↪3 columnas
print("Tipo de dato:", mini_t.dtype)      # torch.float32
```

Como array de NumPy:

```
[[ 0. 128. 255.]
 [ 64. 128. 192.]
 [255. 128.  0.]]
```

Como tensor de PyTorch:

```
tensor([[ 0., 128., 255.],
        [ 64., 128., 192.],
        [255., 128.,  0.]])
```

Forma (shape): torch.Size([3, 3])

Tipo de dato: torch.float32

La forma (shape) es el DNI del tensor. Una red no recibe una imagen suelta de 3×3: recibe un **lote** de imágenes, cada una con sus **canales**. Por eso las imágenes se guardan con **4 dimensiones**: [N, C, H, W].

Letra	Significa	En MNIST
N	cuántas imágenes hay en el lote	64
C	canales (1 = grises, 3 = color RGB)	1
H	alto en píxeles	28
W	ancho en píxeles	28

Veamos cómo nuestra mini imagen se convierte en un tensor de 4 dimensiones.

```
[4]: # unsqueeze(0) agrega una dimensión de tamaño 1 al inicio (no cambia los
      ↪ números, solo la forma).
con_canal = mini_t.unsqueeze(0)          # [3, 3]      -> [1, 3, 3]      (le
      ↪ damos 1 canal)
con_lote = con_canal.unsqueeze(0)        # [1, 3, 3]    -> [1, 1, 3, 3] (un
      ↪ lote de 1 imagen)

print("Imagen sola:           ", mini_t.shape)      # [H, W]
print("Con canal:             ", con_canal.shape)   # [C, H, W]
print("Con lote (lo que pide la red):", con_lote.shape) # [N, C, H, W]

# Normalizar: dividir entre 255 lleva los valores de [0, 255] a [0, 1] (como en
      ↪ la Clase 2).
normalizada = mini_t / 255.0              # cada número se divide entre 255
print("\nNormalizada entre 0 y 1:\n", normalizada)

# Mover el tensor a la GPU (o dejarlo en CPU si no hay GPU). La sintaxis es la
      ↪ misma.
en_dispositivo = normalizada.to(device)    # .to(device) lo lleva a "cuda" o a
      ↪ "cpu"
print("\nEl tensor ahora vive en:", en_dispositivo.device)
```

```
Imagen sola:           torch.Size([3, 3])
Con canal:             torch.Size([1, 3, 3])
Con lote (lo que pide la red): torch.Size([1, 1, 3, 3])
```

```
Normalizada entre 0 y 1:
tensor([[0.0000, 0.5020, 1.0000],
        [0.2510, 0.5020, 0.7529],
        [1.0000, 0.5020, 0.0000]])
```

```
El tensor ahora vive en: cpu
```

2.3 Parte 2 — Una neurona, hecha a mano

Antes de construir una red entera, miremos su pieza mínima: **una neurona**. Una neurona hace tres cosas muy simples:

1. Multiplica cada entrada por un **peso** (cuánto le importa esa entrada).
2. Suma todo y le agrega un **sesgo** (un ajuste fijo).
3. Pasa el resultado por una **activación**. Usaremos **ReLU**: si el número es negativo lo vuelve 0; si es positivo lo deja igual.

Los **pesos** y el **sesgo** son los **parámetros** de la red: como las cantidades de sal, limón o ají en una receta, son los números que se van corrigiendo hasta que el resultado sale bien. Hagámoslo con 4 píxeles.

```
[5]: # Cuatro píxeles de entrada, ya normalizados entre 0 y 1.
x = torch.tensor([0.0, 0.8, 1.0, 0.2])

# Un peso por cada píxel: dicen cuánto "le importa" cada píxel a la neurona.
w = torch.tensor([0.5, -0.3, 0.9, 0.4])

# El sesgo: un ajuste que se suma al final.
b = torch.tensor(0.1)

# Paso 1 y 2: multiplicar píxel por peso, sumar todo y agregar el sesgo.
suma = (x * w).sum() + b          # x * w multiplica elemento a elemento;
    ↪ .sum() los suma
print("Suma ponderada + sesgo:", round(suma.item(), 2))    # .item() saca el
    ↪ número del tensor

# Paso 3: la activación ReLU deja pasar positivos y convierte negativos en 0.
salida = torch.relu(suma)
print("Salida de la neurona (tras ReLU):", round(salida.item(), 2))

# Probemos ReLU con varios números para ver su regla:
prueba = torch.tensor([-2.0, -0.5, 0.0, 0.7, 3.0])
print("\nEntradas a ReLU:", prueba.tolist())
print("Salidas de ReLU: ", torch.relu(prueba).tolist())    # los negativos
    ↪ quedan en 0
```

Suma ponderada + sesgo: 0.84

Salida de la neurona (tras ReLU): 0.84

Entradas a ReLU: [-2.0, -0.5, 0.0, 0.699999988079071, 3.0]

Salidas de ReLU: [0.0, 0.0, 0.0, 0.699999988079071, 3.0]

Lo mismo, con una capa de PyTorch. `nn.Linear` es exactamente esta cuenta (multiplicar por pesos, sumar y agregar sesgo), pero para muchas neuronas a la vez. La diferencia es que sus pesos arrancan **al azar**, y entrenar consiste en ajustarlos.

```
[6]: # nn.Linear(4, 1): una capa que recibe 4 números y entrega 1 (o sea, UNA
    ↪ neurona).
capa = nn.Linear(4, 1)

# Le ponemos a mano los mismos pesos y sesgo de arriba (normalmente NO se hace:
    ↪ se aprenden).
with torch.no_grad():          # no_grad: estamos editando a mano,
    ↪ no entrenando
    capa.weight[:] = w          # los 4 pesos
    capa.bias[:] = b            # el sesgo

resultado = torch.relu(capa(x))    # la capa hace la suma ponderada;
    ↪ luego aplicamos ReLU
```

```

print("Con nn.Linear + ReLU:", round(resultado.item(), 2)) # debe dar lo
↳ mismo que a mano

# Una capa real de nuestro modelo: 784 entradas y 128 neuronas.
capa_real = nn.Linear(784, 128)
print("\nPesos de una capa 784 -> 128:", capa_real.weight.shape) # 128 filas
↳ x 784 columnas
print("Sesgos:", capa_real.bias.shape) # uno por
↳ neurona
print("Parámetros en esa sola capa:", 784 * 128 + 128)

```

Con nn.Linear + ReLU: 0.84

Pesos de una capa 784 -> 128: torch.Size([128, 784])

Sesgos: torch.Size([128])

Parámetros en esa sola capa: 100480

2.4 Parte 3 — El dataset MNIST

Para **entrenar** no basta una imagen: hacen falta **miles** de ejemplos, cada uno con su respuesta correcta (su **etiqueta**). Por eso hoy, y por una buena razón, dejamos a un lado el “sube tu foto” de la Unidad 1: con una sola foto la red no aprende, memoriza.

Usamos **MNIST**: 70 000 dígitos escritos a mano, en grises, de 28×28 píxeles. **torchvision** lo descarga por ti (unos segundos). Tu foto vuelve en la Parte 11: entrenamos con MNIST y probamos con un dígito tuyo.

MNIST viene partido en dos: - **Entrenamiento (60 000)**: los ejercicios con los que la red estudia. - **Prueba (10 000)**: el examen, con dígitos que la red **nunca vio**. Solo así sabemos si aprendió o si memorizó.

```

[7]: # transforms.ToTensor() hace dos cosas con cada imagen:
#    1) la convierte en tensor con forma [1, 28, 28] (1 canal, 28 de alto, 28
↳ de ancho)
#    2) divide los píxeles entre 255, para que queden entre 0 y 1 (la
↳ normalización de la Clase 2)
transformacion = transforms.ToTensor()

# train=True -> las 60 000 imágenes de entrenamiento. download=True -> las
↳ descarga si no están.
train_data = datasets.MNIST(root="./data", train=True, download=True,
↳ transform=transformacion)

# train=False -> las 10 000 imágenes de prueba (el "examen" que la red nunca ve
↳ al estudiar).
test_data = datasets.MNIST(root="./data", train=False, download=True,
↳ transform=transformacion)

```

```
print("Imágenes de entrenamiento:", len(train_data))    # 60000
print("Imágenes de prueba:      ", len(test_data))      # 10000
```

```
100%|      | 9.91M/9.91M [00:02<00:00, 3.85MB/s]
100%|      | 28.9k/28.9k [00:00<00:00, 361kB/s]
100%|      | 1.65M/1.65M [00:00<00:00, 2.52MB/s]
100%|      | 4.54k/4.54k [00:00<00:00, 5.20MB/s]
```

Imágenes de entrenamiento: 60000

Imágenes de prueba: 10000

```
[8]: # Cada elemento del dataset es un par (imagen, etiqueta). Tomamos el primero:
imagen0, etiqueta0 = train_data[0]

print("Forma de la imagen:", imagen0.shape)           # torch.Size([1, 28, 28])
print("Etiqueta (el dígito correcto):", etiqueta0)    # un número del 0 al 9
print("Valor mínimo y máximo de sus píxeles:", imagen0.min().item(), "y",
      ↪imagen0.max().item())

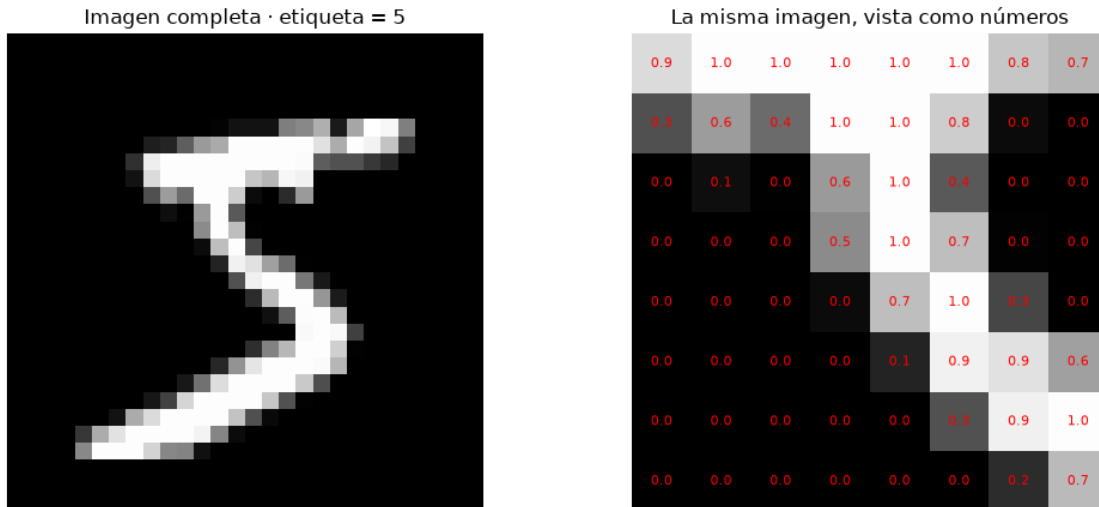
# Mostramos la imagen y, al lado, una zona de 8x8 píxeles en números.
fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
ax[0].imshow(imagen0.squeeze(), cmap="gray")          # squeeze() quita el
      ↪canal: [1,28,28] -> [28,28]
ax[0].set_title(f"Imagen completa · etiqueta = {etiqueta0}")
ax[0].axis("off")

zona = imagen0.squeeze()[8:16, 8:16]                  # filas 8 a 15 y
      ↪columnas 8 a 15
ax[1].imshow(zona, cmap="gray", vmin=0, vmax=1)        # vmin/vmax: 0 = negro,
      ↪1 = blanco
for i in range(8):                                    # recorreremos las 8
      ↪filas...
    for j in range(8):                                # ...y las 8 columnas
        ax[1].text(j, i, f"{zona[i, j]:.1f}", ha="center", va="center",
                    color="red", fontsize=8)           # escribimos el número
      ↪de cada píxel
ax[1].set_title("La misma imagen, vista como números")
ax[1].axis("off")
plt.tight_layout(); plt.show()
```

Forma de la imagen: torch.Size([1, 28, 28])

Etiqueta (el dígito correcto): 5

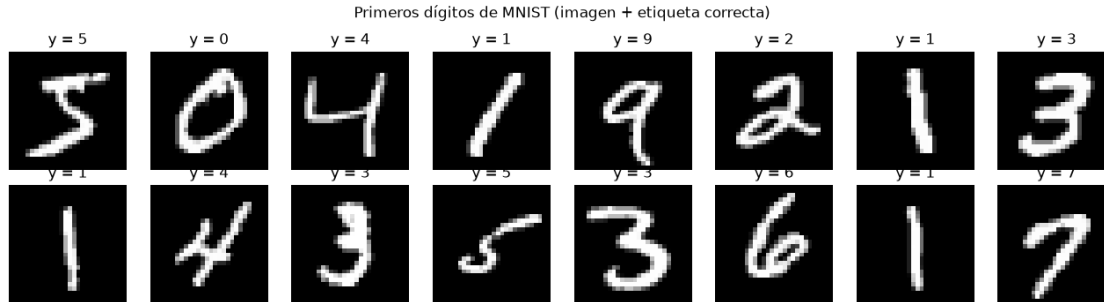
Valor mínimo y máximo de sus píxeles: 0.0 y 1.0



Antes de ejecutar la siguiente celda, predice: ¿los dígitos se verán todos iguales o con caligrafías distintas? Mirar los datos antes de entrenar es un paso que **nunca** se salta.

```
[9]: # Mostramos las primeras 16 imágenes con su etiqueta, en 2 filas de 8.
fig, axes = plt.subplots(2, 8, figsize=(13, 3.6))
for i, ax in enumerate(axes.flat):
    # axes.flat recorre los
    img, etiqueta = train_data[i]
    # imagen y etiqueta
    # número i
    ax.imshow(img.squeeze(), cmap="gray")
    # dibujamos el dígito
    ax.set_title(f"y = {etiqueta}")
    # arriba, su etiqueta
    # real
    ax.axis("off")
    # sin ejes, para que se
    # vea limpio
plt.suptitle("Primeros dígitos de MNIST (imagen + etiqueta correcta)")
plt.tight_layout(); plt.show()

# ¿Cuántos ejemplos hay de cada dígito? (un dataset equilibrado es buena
# noticia)
conteo = torch.bincount(train_data.targets)
# cuenta cuántas veces
# aparece cada etiqueta
for digito, n in enumerate(conteo.tolist()):
    print(f"Dígito {digito}: {n} imágenes")
```



Dígito 0: 5923 imágenes
 Dígito 1: 6742 imágenes
 Dígito 2: 5958 imágenes
 Dígito 3: 6131 imágenes
 Dígito 4: 5842 imágenes
 Dígito 5: 5421 imágenes
 Dígito 6: 5918 imágenes
 Dígito 7: 6265 imágenes
 Dígito 8: 5851 imágenes
 Dígito 9: 5949 imágenes

2.5 Parte 4 — El DataLoader: servir los datos por lotes

La red no estudia las 60 000 imágenes de golpe, sino en **lotes** (en inglés, *batches*) pequeños. Después de cada lote ajusta sus parámetros un poquito, como una pollería que hornea por tandas y ajusta el fuego después de cada tanda. El `DataLoader` se encarga de:

- **agrupar** las imágenes de 64 en 64,
- **barajarlas** en cada vuelta (`shuffle=True`), para que la red no aprenda el orden en vez del contenido.

```
[10]: # batch_size = cuántas imágenes ve el modelo antes de ajustar sus parámetros.
train_loader = DataLoader(train_data, batch_size=64, shuffle=True) #
    ↪ barajamos para estudiar
test_loader = DataLoader(test_data, batch_size=256, shuffle=False) # el
    ↪ examen no hace falta barajarlo

# next(iter(...)) pide UN lote al DataLoader, para mirarlo.
imagenes, etiquetas = next(iter(train_loader))

print("Forma de un lote de imágenes:", imagenes.shape) # [64, 1, 28, 28] ->
    ↪ [N, C, H, W]
print("Forma de las etiquetas:      ", etiquetas.shape) # [64] -> una etiqueta
    ↪ por imagen
```

```
print("Primeras 10 etiquetas del lote:", etiquetas[:10].tolist())    # están
    ↪ barajadas

# ¿Cuántos lotes hay en una vuelta completa (una "época")?
print("\nLotes por época:", len(train_loader))    # 60000 / 64 = 937.5 -> 938
    ↪ lotes
```

Forma de un lote de imágenes: `torch.Size([64, 1, 28, 28])`
 Forma de las etiquetas: `torch.Size([64])`
 Primeras 10 etiquetas del lote: `[0, 8, 5, 8, 2, 8, 3, 5, 0, 3]`

Lotes por época: 938

2.6 Parte 5 — El modelo: un perceptrón multicapa (MLP)

Ahora juntamos muchas neuronas en **capas**. Nuestro MLP hace tres cosas:

1. **Aplana** la imagen: convierte la rejilla `[1, 28, 28]` en una fila de 784 números. (*Recuerda este paso: es el protagonista del final.*)
2. Pasa esa fila por **dos capas densas** (Linear) con **ReLU** entre ellas: $784 \rightarrow 128 \rightarrow 64$.
3. Entrega **10 números** (los *logits*), uno por dígito. El más alto es la predicción.

Primero veamos el aplanado solo, para entenderlo.

```
[11]: # nn.Flatten() convierte cada imagen [1, 28, 28] en una fila de 784 números.
aplanar = nn.Flatten()
lote_plano = aplanar(imagenes)    # aplicamos a las 64 imágenes
    ↪ del lote

print("Antes de aplanar: ", imagenes.shape)    # [64, 1, 28, 28]
print("Después de aplanar:", lote_plano.shape)    # [64, 784] -> 64 filas de
    ↪ 784 números
print("28 x 28 =", 28 * 28)    # de ahí sale el 784
```

Antes de aplanar: `torch.Size([64, 1, 28, 28])`
 Después de aplanar: `torch.Size([64, 784])`
`28 x 28 = 784`

```
[12]: # Definimos el modelo como una "clase": una plantilla con sus capas y cómo se
    ↪ usan.
class MLP(nn.Module):    # nn.Module es la base de
    ↪ todo modelo en PyTorch
    def __init__(self):    # __init__: aquí se CREAN las
    ↪ capas
        super().__init__()    # línea obligatoria: prepara
    ↪ el modelo por dentro
        self.aplanar = nn.Flatten()    # [N,1,28,28] -> [N,784]
```

```

        self.capas = nn.Sequential(
            ↪fila, una tras otra
            nn.Linear(28 * 28, 128),
            ↪neuronas
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10),
            ↪(un número por dígito)
        )

        def forward(self, x):
            ↪imagen por el modelo
            x = self.aplanar(x)
            return self.capas(x)
            ↪salen 10 logits

model = MLP().to(device)
    ↪llevamos a GPU/CPU
print(model)

```

Sequential: las capas en

capa 1: 784 entradas -> 128

activación: negativos a cero

capa 2: 128 -> 64 neuronas

activación otra vez

capa de salida: 64 -> 10

forward: cómo VIAJA la

primero aplanamos

luego pasa por las capas y

creamos el modelo y lo

muestra la arquitectura

```

MLP(
    (aplanar): Flatten(start_dim=1, end_dim=-1)
    (capas): Sequential(
      (0): Linear(in_features=784, out_features=128, bias=True)
      (1): ReLU()
      (2): Linear(in_features=128, out_features=64, bias=True)
      (3): ReLU()
      (4): Linear(in_features=64, out_features=10, bias=True)
    )
)

```

```

[13]: # Contemos los parámetros capa por capa: pesos + sesgos.
capa1 = 784 * 128 + 128      # 784 pesos por cada una de las 128 neuronas, más
    ↪128 sesgos
capa2 = 128 * 64 + 64        # 128 pesos por cada una de las 64 neuronas, más
    ↪64 sesgos
capa3 = 64 * 10 + 10         # 64 pesos por cada una de las 10 salidas, más 10
    ↪sesgos
print("Capa 1:", capa1, "| Capa 2:", capa2, "| Capa 3:", capa3)
print("Total a mano:  ", capa1 + capa2 + capa3)

# PyTorch los cuenta solo: p.numel() = cuántos números tiene cada bloque de
    ↪parámetros.
total = sum(p.numel() for p in model.parameters())
print("Total con PyTorch:", total)
    ↪parámetros

```

deben coincidir: 109 386

Capa 1: 100480 | Capa 2: 8256 | Capa 3: 650
Total a mano: 109386
Total con PyTorch: 109386

De logits a probabilidades. Los 10 números que salen del modelo no están en ninguna escala. La función **softmax** los convierte en **probabilidades** que suman 1. Y como el modelo todavía no ha entrenado (sus parámetros están al azar), **predice como quien lanza un dado de 10 caras**. Predice antes de ejecutar: ¿qué exactitud tendrá?

```
[14]: model.eval()                                # modo evaluación (solo
      ↪predecir, no entrenar)
      with torch.no_grad():                        # no_grad: no guardamos
      ↪historial de cálculo (más rápido)
          logits = model(imagenes[:1].to(device))  # predécimos UNA imagen -> 10
      ↪logits
          probs = torch.softmax(logits, dim=1)      # softmax: logits ->
      ↪probabilidades que suman 1

      print("Logits (sin escala): ", [round(v, 2) for v in logits[0].tolist()])
      print("Probabilidades:      ", [round(v, 3) for v in probs[0].tolist()])
      print("Suman:", round(probs.sum().item(), 4)) # siempre 1
      print("Predicción (el más alto):", probs.argmax().item(), "| Etiqueta real:",
      ↪etiquetas[0].item())

      # Ahora medimos la exactitud del modelo SIN entrenar sobre un lote del examen:
      with torch.no_grad():
          img_test, lab_test = next(iter(test_loader)) # 256
      ↪imágenes de prueba
          predicciones = model(img_test.to(device)).argmax(dim=1).cpu() # el dígito
      ↪con mayor puntaje
      exactitud_inicial = (predicciones == lab_test).float().mean().item() #
      ↪proporción de aciertos
      print(f"\nExactitud SIN entrenar: {exactitud_inicial*100:.1f}% (se espera
      ↪cerca de 10%: puro azar)")
```

```
Logits (sin escala): [0.01, 0.09, -0.11, 0.08, 0.13, -0.0, -0.07, -0.13,
0.05, -0.01]
Probabilidades: [0.1, 0.108, 0.089, 0.108, 0.113, 0.099, 0.093, 0.087,
0.105, 0.098]
Suman: 1.0
Predicción (el más alto): 4 | Etiqueta real: 0
```

Exactitud SIN entrenar: 14.5% (se espera cerca de 10%: puro azar)

2.7 Parte 6 — La pérdida: medir cuánto se equivoca

Para mejorar, el modelo necesita un número que diga **qué tan mal** lo hizo: la **pérdida** (*loss*). Para clasificar usamos la **entropía cruzada** (CrossEntropyLoss). Su idea es simple: mira qué probabilidad le dio el modelo a la **respuesta correcta**.

- Si le dio mucha (0.9) → pérdida **baja**.
- Si le dio poca (0.02) → pérdida **alta**.

La fórmula es **pérdida** = $-\log(\text{probabilidad de la clase correcta})$. Probémosla con un ejemplo pequeño de **3 clases** (gato, perro, pájaro), donde la respuesta correcta es **gato**.

```
[15]: import math                                     # para calcular logaritmos a
      ↪mano

# Tres situaciones: probabilidades que el modelo dio a [gato, perro, pájaro].
      ↪La correcta es gato.
casos = {
    "Seguro y acierta":      [0.90, 0.05, 0.05],
    "Duda entre los tres":   [0.34, 0.33, 0.33],
    "Seguro y se equivoca":  [0.02, 0.95, 0.03],
}
for nombre, p in casos.items():
    perdida = -math.log(p[0])                          # p[0] = probabilidad que le
      ↪dio a "gato"
    print(f"{nombre:22s} -> prob. de gato {p[0]:.2f} -> pérdida {perdida:.2f}")

# ¿Qué pérdida tiene un modelo que reparte 1/10 a cada dígito? (nuestro modelo
      ↪sin entrenar)
print("\nPérdida de adivinar al azar entre 10 clases:", round(-math.log(0.1),
      ↪2))      # 2.30
```

Seguro y acierta	-> prob. de gato 0.90 -> pérdida 0.11
Duda entre los tres	-> prob. de gato 0.34 -> pérdida 1.08
Seguro y se equivoca	-> prob. de gato 0.02 -> pérdida 3.91

Pérdida de adivinar al azar entre 10 clases: 2.3

Guarda ese **2.30**: es la pérdida con la que va a arrancar el entrenamiento. Ahora comprobemos que `nn.CrossEntropyLoss` de PyTorch calcula lo mismo. Ojo: PyTorch recibe **logits** (no probabilidades) y aplica el softmax por dentro.

```
[16]: criterion = nn.CrossEntropyLoss()                # la pérdida que usaremos
      ↪para entrenar

# Logits de ejemplo para 3 clases y la etiqueta correcta (clase 0 = gato).
logits_ejemplo = torch.tensor([[2.5, 0.3, -1.0]])     # el modelo le da más
      ↪puntaje a la clase 0
```

```

etiqueta_ok = torch.tensor([0]) # la respuesta correcta es
↳ la clase 0

p_gato = torch.softmax(logits_ejemplo, dim=1)[0, 0].item() # probabilidad de
↳ "gato" tras softmax
print("Probabilidad de gato:", round(p_gato, 3))
print("Pérdida a mano: ", round(-math.log(p_gato), 4))
print("Pérdida PyTorch: ", round(criterion(logits_ejemplo, etiqueta_ok).item(),
↳ 4)) # la misma

# Con nuestro MLP sin entrenar, sobre un lote real:
with torch.no_grad():
    perdida_inicial = criterion(model(imagenes.to(device)), etiquetas.
↳ to(device)).item()
print("\nPérdida del MLP sin entrenar:", round(perdida_inicial, 2), "(cerca de
↳ 2.30, como predijimos)")

```

Probabilidad de gato: 0.876

Pérdida a mano: 0.1319

Pérdida PyTorch: 0.1319

Pérdida del MLP sin entrenar: 2.31 (cerca de 2.30, como predijimos)

2.8 Parte 7 — El gradiente: cómo sabe hacia dónde corregir

Ya medimos el error. ¿Cómo sabe la red **qué parámetro mover y hacia dónde?** Con el **gradiente**: la pendiente de la pérdida respecto a cada parámetro. Si la pendiente es negativa, conviene subir el parámetro; si es positiva, bajarlo.

Es como **buscar señal en el celular**: no sabes dónde está la antena, pero das un paso, miras si las barritas suben o bajan, y te mueves hacia donde mejora. El gradiente es esa dirección, y el tamaño de cada paso es el **learning rate** (tasa de aprendizaje), como la pizca de sal que echas antes de volver a probar.

Lo hacemos con **un solo parámetro** w . Queremos que $w \times 2$ dé 6, así que la respuesta ideal es $w = 3$. Arrancamos en $w = 0$.

```

[17]: # requires_grad=True le pide a PyTorch que RECUERDE cómo se usa w, para
↳ calcular su gradiente.

w = torch.tensor(0.0, requires_grad=True) # nuestro único parámetro,
↳ arranca en 0

x_ej, y_ej = 2.0, 6.0 # entrada y respuesta
↳ correcta

lr = 0.05 # learning rate: el tamaño
↳ del paso

for paso in range(8): # damos 8 pasos cuesta abajo

```

```

    prediccion = w * x_ej                                # el "modelo": multiplicar
    ↪por w
    perdida = (prediccion - y_ej) ** 2                    # error al cuadrado
    ↪(siempre positivo)
    perdida.backward()                                    # BACKWARD: PyTorch calcula
    ↪el gradiente de w
    print(f"paso {paso}: w = {w.item():.3f} | pérdida = {perdida.item():6.2f} |
    ↪gradiente = {w.grad.item():7.2f}")
    with torch.no_grad():                                # vamos a mover w "a mano",
    ↪sin registrarlo
        w -= lr * w.grad                                  # PASO: w se mueve en
    ↪contra de la pendiente
        w.grad.zero_()                                    # RESET: borramos el
    ↪gradiente para el siguiente paso

print("\nw final:", round(w.item(), 3), "-> se acerca a 3, la respuesta ideal")

```

```

paso 0: w = 0.000 | pérdida = 36.00 | gradiente = -24.00
paso 1: w = 1.200 | pérdida = 12.96 | gradiente = -14.40
paso 2: w = 1.920 | pérdida = 4.67 | gradiente = -8.64
paso 3: w = 2.352 | pérdida = 1.68 | gradiente = -5.18
paso 4: w = 2.611 | pérdida = 0.60 | gradiente = -3.11
paso 5: w = 2.767 | pérdida = 0.22 | gradiente = -1.87
paso 6: w = 2.860 | pérdida = 0.08 | gradiente = -1.12
paso 7: w = 2.916 | pérdida = 0.03 | gradiente = -0.67

```

w final: 2.95 -> se acerca a 3, la respuesta ideal

Fíjate en el patrón: gradiente grande y negativo al inicio (lejos del ideal, conviene subir mucho), y cada vez más chico a medida que w se acerca a 3. Eso mismo pasará con los 109 386 parámetros del MLP, todos a la vez.

En la práctica no movemos los parámetros a mano: lo hace un **optimizador**. Usaremos **SGD** (descenso de gradiente estocástico), que hace exactamente $\text{parámetro} -= \text{lr} \times \text{gradiente}$ para todos.

```

[18]: # El optimizador recibe TODOS los parámetros del modelo y el tamaño de paso.
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

print(optimizer)    # muestra la configuración: SGD con lr = 0.1

```

```

SGD (
Parameter Group 0
  dampening: 0
  differentiable: False
  foreach: None
  fused: None
  lr: 0.1
  maximize: False

```

```

momentum: 0
nesterov: False
weight_decay: 0
)

```

2.9 Parte 8 — El loop de entrenamiento

Aquí se junta todo. Para **cada lote** se repiten los mismos cinco pasos, los mismos que hicimos con el parámetro `w`:

Paso	Código	Qué hace
1	<code>salida = model(X)</code>	Forward: el modelo predice
2	<code>perdida = criterion(salida, y)</code>	Pérdida: mide el error
3	<code>perdida.backward()</code>	Backward: calcula el gradiente de cada parámetro
4	<code>optimizer.step()</code>	Paso: ajusta todos los parámetros
5	<code>optimizer.zero_grad()</code>	Reset: limpia los gradientes para el siguiente lote

Una **época** es una vuelta completa a los 938 lotes. Haremos 5 épocas: 4 690 correcciones. Primero una función para **evaluar** en el examen (sin entrenar).

```

[19]: def evaluar(loader):
    """Recorre un DataLoader y devuelve (pérdida promedio, exactitud). NO
    ↪entrena."""
    model.eval() # modo evaluación
    perdida_total, aciertos, total = 0.0, 0, 0 # acumuladores
    with torch.no_grad(): # sin gradientes: solo
    ↪medimos
        for X, y in loader: # recorreremos todos los lotes
            X, y = X.to(device), y.to(device) # los llevamos a GPU/CPU
            salida = model(X) # predicción: 10 logits por
    ↪imagen
            perdida_total += criterion(salida, y).item() * X.size(0) #
    ↪pérdida del lote x tamaño
            aciertos += (salida.argmax(1) == y).sum().item() #
    ↪cuántas acertó
            total += X.size(0) # cuántas imágenes llevamos
        return perdida_total / total, aciertos / total # promedios finales

print("Función evaluar lista.")

```

Función evaluar lista.

Antes de ejecutar, predice: ¿con qué pérdida arrancará la primera época y cuánto subirá la exactitud en esa primera vuelta? El entrenamiento tarda entre uno y tres minutos. Mientras corre, relee los cinco pasos de la tabla.

```
[20]: EPOCAS = 5                                # cuántas vueltas
      ↪ completas daremos
historial = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc":
      ↪ []} # para las curvas

for epoca in range(1, EPOCAS + 1):              # epoca = 1, 2, 3, 4, 5
    model.train()                               # modo entrenamiento
    perdida_total, aciertos, total = 0.0, 0, 0   # acumuladores de esta
    ↪ época

    for X, y in train_loader:                   # recorremos los 938 lotes
        X, y = X.to(device), y.to(device)      # a GPU/CPU

        optimizer.zero_grad()                  # 5) RESET: borramos
    ↪ gradientes del lote anterior
        salida = model(X)                      # 1) FORWARD: el modelo
    ↪ predice
        perdida = criterion(salida, y)          # 2) PÉRDIDA: cuánto se
    ↪ equivocó
        perdida.backward()                     # 3) BACKWARD: gradiente
    ↪ de los 109 386 parámetros
        optimizer.step()                      # 4) PASO: ajusta todos
    ↪ los parámetros

        perdida_total += perdida.item() * X.size(0) # sumamos la
    ↪ pérdida del lote
        aciertos += (salida.argmax(1) == y).sum().item() # sumamos los
    ↪ aciertos del lote
        total += X.size(0)                    # sumamos
    ↪ cuántas imágenes vimos

    tr_loss, tr_acc = perdida_total / total, aciertos / total # promedios
    ↪ del entrenamiento
    te_loss, te_acc = evaluar(test_loader)      # el "examen"
    ↪ de esta época

    historial["train_loss"].append(tr_loss); historial["train_acc"].
    ↪ append(tr_acc) # guardamos
    historial["test_loss"].append(te_loss); historial["test_acc"].
    ↪ append(te_acc)
```

```
print(f"Época {epoca}/{EPOCAS} | entrenamiento: pérdida {tr_loss:.3f},  
↪ exactitud {tr_acc*100:5.2f}% "  
      f"| prueba: pérdida {te_loss:.3f}, exactitud {te_acc*100:5.2f}%")
```

Época 1/5 | entrenamiento: pérdida 0.509, exactitud 85.38% | prueba: pérdida 0.228, exactitud 93.22%
 Época 2/5 | entrenamiento: pérdida 0.193, exactitud 94.41% | prueba: pérdida 0.150, exactitud 95.63%
 Época 3/5 | entrenamiento: pérdida 0.134, exactitud 95.98% | prueba: pérdida 0.120, exactitud 96.25%
 Época 4/5 | entrenamiento: pérdida 0.103, exactitud 96.92% | prueba: pérdida 0.107, exactitud 96.80%
 Época 5/5 | entrenamiento: pérdida 0.084, exactitud 97.51% | prueba: pérdida 0.097, exactitud 97.07%

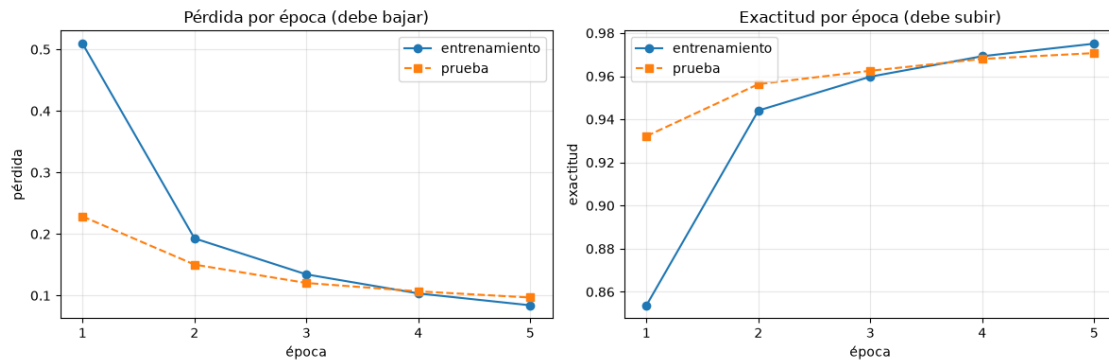
Lee la tabla como una historia. La primera época hace el salto grande: la exactitud pasa del ~10 % a más del 90 %, porque el modelo venía de cero. Las siguientes mejoran cada vez menos: quedan solo ajustes finos. Deberías terminar cerca de **97 %** en prueba.

2.10 Parte 9 — Pérdida y exactitud como curvas

Los números se leen mejor dibujados. Estas dos curvas son el **electrocardiograma** del modelo: de un vistazo dicen si aprende, si se estancó o (lo veremos en la Clase 7) si empieza a memorizar.

```
[21]: epocas = range(1, EPOCAS + 1) # eje horizontal: 1, 2, 3, 4, 5
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4)) # dos gráficos lado a lado
# Izquierda: la pérdida, que debe BAJAR.
ax1.plot(epocas, historial["train_loss"], "o-", label="entrenamiento")
ax1.plot(epocas, historial["test_loss"], "s--", label="prueba")
ax1.set_title("Pérdida por época (debe bajar)")
ax1.set_xlabel("época"); ax1.set_ylabel("pérdida"); ax1.legend(); ax1.grid(alpha=0.3)
ax1.set_xticks(list(epocas)) # solo épocas enteras en el eje: 1, 2, 3...
# Derecha: la exactitud, que debe SUBIR.
ax2.plot(epocas, historial["train_acc"], "o-", label="entrenamiento")
ax2.plot(epocas, historial["test_acc"], "s--", label="prueba")
ax2.set_title("Exactitud por época (debe subir)")
ax2.set_xlabel("época"); ax2.set_ylabel("exactitud"); ax2.legend(); ax2.grid(alpha=0.3)
ax2.set_xticks(list(epocas))
```

```
plt.tight_layout(); plt.show()
```



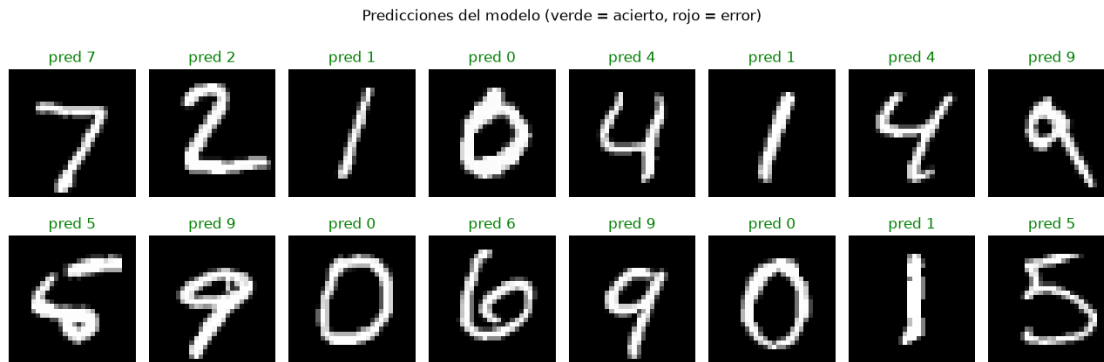
Qué mirar: las dos curvas se aplanan hacia el final (el modelo ya aprendió casi todo lo que puede), y las líneas de entrenamiento y prueba van **juntas**. Si la de prueba empezara a empeorar mientras la de entrenamiento sigue mejorando, el modelo estaría memorizando: eso se llama **sobreajuste** y es el tema de la Clase 7.

2.11 Parte 10 — Ver al modelo en acción (y sus errores)

Números y curvas están bien, pero veamos **predicciones reales** del examen. En **verde** los aciertos, en **rojo** los errores. Después miramos solo los errores: suelen ser dígitos que hasta a una persona le costaría leer.

```
[22]: model.eval() # modo evaluación
img_test, lab_test = next(iter(test_loader)) # un lote de 256 imágenes
    # del examen
with torch.no_grad():
    pred_test = model(img_test.to(device)).argmax(1).cpu() # el dígito
    # predicho para cada una

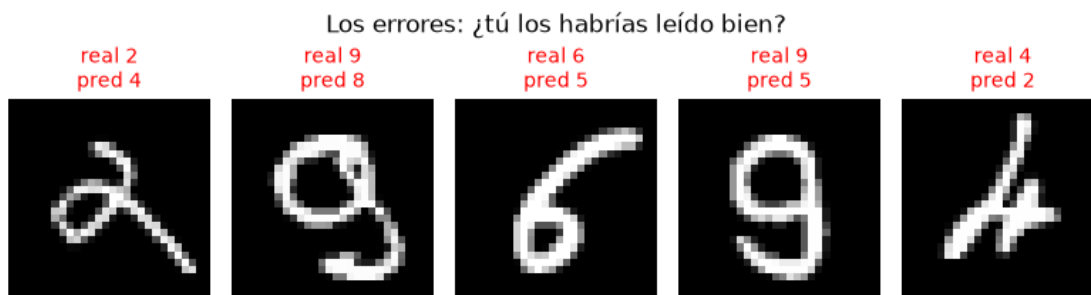
fig, axes = plt.subplots(2, 8, figsize=(13, 4.6))
for i, ax in enumerate(axes.flat): # las primeras 16
    ax.imshow(img_test[i].squeeze(), cmap="gray")
    ok = pred_test[i] == lab_test[i] # True si acertó
    ax.set_title(f"pred {pred_test[i].item()}", color="green" if ok else "red")
    ax.axis("off")
plt.suptitle("Predicciones del modelo (verde = acierto, rojo = error)")
plt.tight_layout(); plt.show()
```



```
[23]: # Buscamos SOLO los errores dentro de ese lote.
errores = (pred_test != lab_test).nonzero().flatten() # posiciones donde se
↪equivocó
print(f"Errores en este lote: {len(errores)} de {len(lab_test)}")

n = min(8, len(errores)) # mostramos hasta 8
if n > 0:
    fig, axes = plt.subplots(1, n, figsize=(1.6 * n, 2.4), squeeze=False)
    for k in range(n):
        i = errores[k].item() # posición del error
        axes[0, k].imshow(img_test[i].squeeze(), cmap="gray")
        axes[0, k].set_title(f"real {lab_test[i].item()}\npred {pred_test[i].
↪item()}", color="red", fontsize=10)
        axes[0, k].axis("off")
    plt.suptitle("Los errores: ¿tú los habrías leído bien?")
    plt.tight_layout(); plt.show()
```

Errores en este lote: 5 de 256



2.12 Parte 11 — Ahora, tu propio dígito

Recuperamos el flujo de la Unidad 1: **sube una imagen tuya** de un dígito. Escríbelo **grueso**, con plumón, sobre papel blanco, y tómale una foto de cerca (o dibújalo en Paint). El modelo, que solo estudió MNIST, intentará reconocerlo.

MNIST son dígitos **blancos sobre fondo negro**, centrados, de 28×28 . El código prepara tu foto para que se parezca: grises, invertir si el fondo es claro, recortar el trazo, reducirlo y centrarlo por su centro de masa, igual que la pizarra del laboratorio en vivo.

```
[29]: # -----  
# SUBIDA DE ARCHIVO EN Jupyter  
# 'google.colab.files' solo existe en Colab; por eso lo importamos aquí.  
# -----  
#Se cambio para que funcione con JUPYTER  
ruta = "../Recursos/numero_3.jpeg"  
digito_bgr = cv2.imread(ruta, cv2.IMREAD_COLOR)  
if digito_bgr is None:  
    raise FileNotFoundError(f"No se pudo leer la imagen: {ruta}")  
nombre = ruta.split("/")[-1]  
print("Imagen cargada:", nombre, "| tamaño:", digito_bgr.shape)
```

Imagen cargada: numero_3.jpeg | tamaño: (1600, 1200, 3)

```
[30]: assert digito_bgr is not None, "Primero sube una imagen en la celda de arriba."  
  
# 1) A escala de grises: MNIST no tiene color.  
gris = cv2.cvtColor(digito_bgr, cv2.COLOR_BGR2GRAY)  
  
# 2) Si el fondo es claro (papel blanco), invertimos: MNIST es trazo blanco  
    ↪ sobre fondo negro.  
if gris.mean() > 127:                                # promedio alto = imagen  
    ↪ mayormente clara                                # invertir: lo blanco pasa a  
    gris = 255 - gris                                  ↪ negro y viceversa  
  
# 3) Limpiamos el fondo con Otsu (Clase 4): el trazo queda blanco puro y el  
    ↪ fondo negro puro.  
_, binaria = cv2.threshold(gris, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)  
  
# 4) Recortamos alrededor del trazo, para que el dígito no quede diminuto en  
    ↪ una esquina.  
filas, cols = np.nonzero(binaria)                    # coordenadas de los píxeles  
    ↪ blancos (el trazo)  
recorte = binaria[filas.min():filas.max() + 1, cols.min():cols.max() + 1]  
  
# 5) Lo metemos en un cuadrado de 20x20 (sin deformarlo) y lo centramos en  
    ↪ 28x28, como MNIST.
```

```

alto, ancho = recorte.shape
escala = 20.0 / max(alto, ancho) # que el lado mayor mida 20
    ↪ píxeles
chico = cv2.resize(recorte, (max(1, int(ancho * escala)), max(1, int(alto *
    ↪ escala))),
                    interpolation=cv2.INTER_AREA)
lienzo = np.zeros((28, 28), dtype=np.uint8) # fondo negro de 28x28
y0 = (28 - chico.shape[0]) // 2 # margen de arriba para centrar
x0 = (28 - chico.shape[1]) // 2 # margen de la izquierda para
    ↪ centrar
lienzo[y0:y0 + chico.shape[0], x0:x0 + chico.shape[1]] = chico

# 5b) Afinamos el centrado por el "centro de masa" del trazo, que es como se
    ↪ preparó MNIST
# (el dashboard de la clase hace exactamente lo mismo).
filas2, cols2 = np.nonzero(lienzo) # píxeles con trazo
pesos_px = lienzo[filas2, cols2].astype(float) # su intensidad
cy = (filas2 * pesos_px).sum() / pesos_px.sum() # fila promedio del
    ↪ trazo
cx = (cols2 * pesos_px).sum() / pesos_px.sum() # columna promedio del
    ↪ trazo
M = np.float32([[1, 0, round(13.5 - cx)], [0, 1, round(13.5 - cy)]]) #
    ↪ desplazamiento hacia el centro
lienzo = cv2.warpAffine(lienzo, M, (28, 28)) # movemos el dígito sin
    ↪ deformarlo

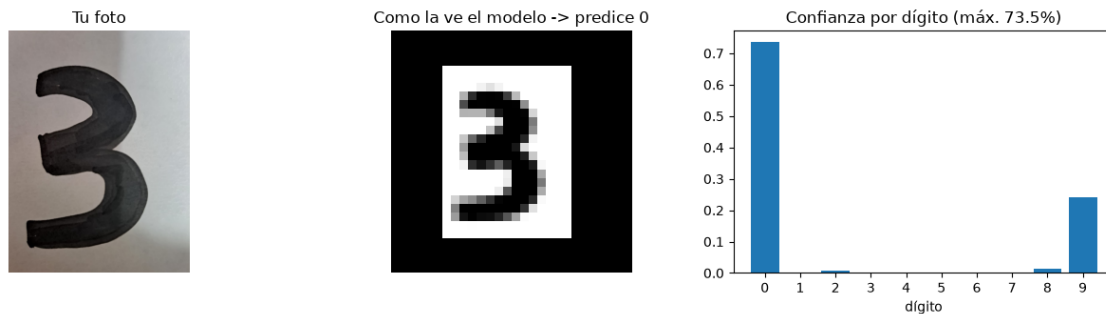
# 6) A tensor con forma [1, 1, 28, 28] y valores entre 0 y 1: exactamente lo
    ↪ que espera el modelo.
tensor_digito = torch.from_numpy(lienzo).float().unsqueeze(0).unsqueeze(0) /
    ↪ 255.0

# 7) Predicción y probabilidades.
model.eval()
with torch.no_grad():
    probs = torch.softmax(model(tensor_digito.to(device)), dim=1)[0].cpu() #
    ↪ 10 probabilidades
pred = probs.argmax().item() # el dígito más probable

# 8) Mostramos tu foto, cómo la ve el modelo y su confianza en cada dígito.
fig, ax = plt.subplots(1, 3, figsize=(13, 3.5))
ax[0].imshow(cv2.cvtColor(digito_bgr, cv2.COLOR_BGR2RGB)); ax[0].set_title("Tu
    ↪ foto"); ax[0].axis("off")
ax[1].imshow(lienzo, cmap="gray"); ax[1].set_title(f"Como la ve el modelo ->
    ↪ predice {pred}"); ax[1].axis("off")
ax[2].bar(range(10), probs.numpy()); ax[2].set_xticks(range(10))

```

```
ax[2].set_title(f"Confianza por dígito (máx. {probs[pred]*100:.1f}%"); ax[2].
    ↪set_xlabel("dígito")
plt.tight_layout(); plt.show()
```



Si falla con tu dígito, no es un error del código. El modelo solo estudió la caligrafía de MNIST; tu trazo, tu grosor y tu cámara pueden ser muy distintos. Ese salto entre *los datos con que se entrena* y *el mundo real* se llama **generalización**, y es uno de los grandes temas del curso.

2.12.1 ¿Y si subes algo que no es un dígito?

Prueba subir la foto de un paisaje, una mascota o tu cara. El modelo **igual** va a responder un número del 0 al 9, y muchas veces con una confianza altísima (en una prueba, un paisaje salió como un **7 con 99 %** de seguridad).

¿Por qué? Porque softmax **siempre** reparte el 100 % entre las 10 clases que conoce. El modelo no tiene una opción “esto no es un dígito”: está obligado a elegir, y elige la que más se parece, aunque ninguna se parezca.

La lección: una confianza alta **no** garantiza una respuesta correcta. Un modelo solo sabe responder dentro del mundo que vio al entrenar; fuera de él, puede equivocarse con total seguridad. Es uno de los riesgos más serios cuando la IA sale del laboratorio.

2.13 Parte 12 — El puente: por qué necesitamos convolución

Nuestro MLP funciona, pero esconde un problema. Su primer paso, **Flatten**, pone los 784 píxeles **en fila**. Al hacerlo pierde la noción de **vecindad**: para el MLP, dos píxeles vecinos no están más relacionados que dos píxeles en esquinas opuestas.

El experimento: desordenamos los píxeles de **todas** las imágenes con la **misma** mezcla fija. Para una persona, los dígitos se vuelven ruido. Entrenamos un MLP nuevo con esas imágenes revueltas y comparamos. **Predice antes:** ¿aprenderá peor, igual o mejor?

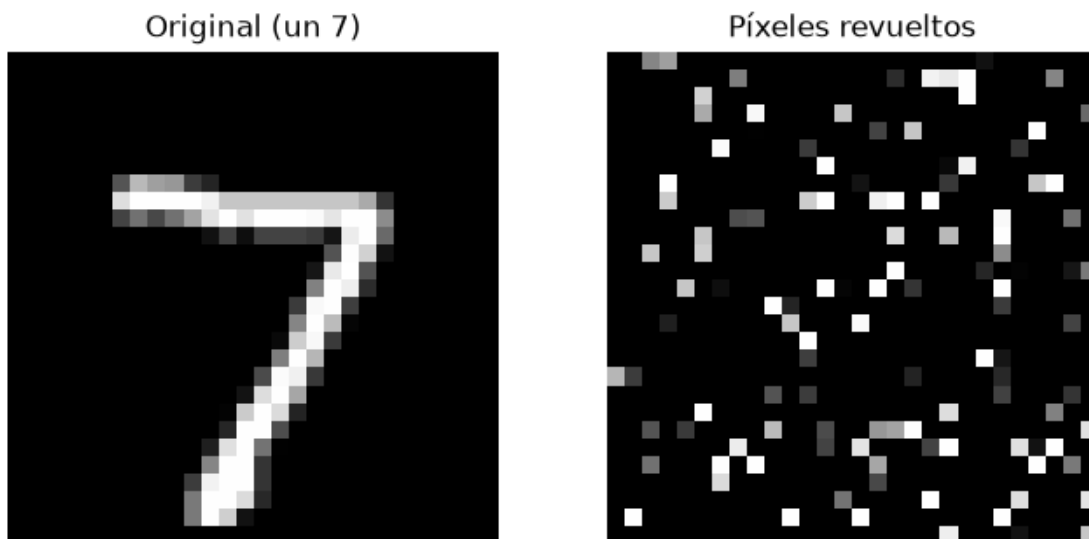
```
[31]: # Una mezcla fija de las 784 posiciones (la misma para todas las imágenes).
perm = torch.randperm(28 * 28)
```

```

# Una transformación que, después de ToTensor, reordena los píxeles con esa
↳ mezcla.
revolver = transforms.Compose([
    transforms.ToTensor(),                                # imagen ->
    ↳ tensor [1,28,28] entre 0 y 1
    transforms.Lambda(lambda t: t.view(-1)[perm].view(1, 28, 28)), # aplanar,
    ↳ revolver, volver a 28x28
])

# Mostramos un dígito original y el mismo dígito revuelto.
original, et = test_data[0]                                # imagen normal
revuelto = revolver(transforms.ToPILImage()(original))      # la misma,
↳ revuelta
fig, ax = plt.subplots(1, 2, figsize=(7, 3.3))
ax[0].imshow(original.squeeze(), cmap="gray"); ax[0].set_title(f"Original (un
↳ {et})"); ax[0].axis("off")
ax[1].imshow(revuelto.squeeze(), cmap="gray"); ax[1].set_title("Píxeles
↳ revueltos"); ax[1].axis("off")
plt.tight_layout(); plt.show()

```



[32]:

```

# Datasets con los píxeles revueltos (mismas imágenes, mismo orden de mezcla
↳ para todas).
train_rev = datasets.MNIST(root="./data", train=True, download=True,
↳ transform=revolver)
test_rev = datasets.MNIST(root="./data", train=False, download=True,
↳ transform=revolver)
train_rev_loader = DataLoader(train_rev, batch_size=64, shuffle=True)

```

```

test_rev_loader = DataLoader(test_rev, batch_size=256, shuffle=False)

modelo_original = model                                # guardamos el modelo bueno
↳para no perderlo
model = MLP().to(device)                               # un MLP NUEVO, desde cero
↳(evaluar() usa 'model')
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

for epoca in range(1, 3):                             # solo 2 épocas, para comparar
↳rápido
    model.train()
    for X, y in train_rev_loader:                     # mismo loop de 5 pasos de la
↳Parte 8
        X, y = X.to(device), y.to(device)
        optimizer.zero_grad()
        perdida = criterion(model(X), y)
        perdida.backward()
        optimizer.step()
    _, acc_rev = evaluar(test_rev_loader)
    print(f"Época {epoca} con píxeles revueltos -> exactitud en prueba:
↳{acc_rev*100:.2f}%")

print(f"\nMLP normal tras 2 épocas:    {historial['test_acc'][1]*100:.2f}%")
print(f"MLP revuelto tras 2 épocas:    {acc_rev*100:.2f}%")
model = modelo_original                               # recuperamos el modelo bueno

```

Época 1 con píxeles revueltos -> exactitud en prueba: 92.75%

Época 2 con píxeles revueltos -> exactitud en prueba: 95.39%

MLP normal tras 2 épocas: 95.63%

MLP revuelto tras 2 épocas: 95.39%

Aprende prácticamente igual. Tú ya no reconoces esos dígitos revueltos; al MLP le da lo mismo, porque **nunca supo quién era vecino de quién**. Y esa vecindad —píxeles juntos que forman bordes, esquinas y trazos— es justo lo que en la Unidad 1 aprovechabas con filtros hechos a mano.

La **red convolucional (CNN)** de la próxima clase no aplanar: desliza filtros pequeños sobre la imagen, la misma convolución de la Clase 3, pero **aprendiendo los números de esos filtros** en vez de diseñarlos. Así recupera la vecindad. Si revolvieras los píxeles, a una CNN sí le dolería: ese será el contraste de la Clase 6.

2.14 Parte 13 — TU TURNO: rompe el modelo tú mismo

En clase viste en el laboratorio en vivo qué le pasa al modelo cuando cambias **una sola línea**. Aquí lo reproduces tú. El código ya está completo: cambia **un valor a la vez**, ejecuta y compara tus curvas con las del modelo original (Parte 9).

Interruptor	Valor normal	Qué probar	Lo que viste en el laboratorio
LR	0.1	0.001 y 1.0	0.001 se queda cerca del 56 %; 1.0 se tambalea y termina algo por debajo
USAR_RELU	True	False	se estanca cerca del 92 %
LIMPIAR_GRADIENTES	True	False	no aprende nada, la pérdida sube
NORMALIZAR	True	False	se rompe en el primer lote y responde siempre "1"
OCULTAS	128	16 y 256	(no lo vimos en clase: descúbrelo tú)

Con EPOCAS_TT = 3 tus números saldrán un poco por debajo de los del laboratorio, que usó 5 épocas.

```
[38]: # =====
# TU TURNO: cambia UN valor a la vez, ejecuta y compara las curvas.
# =====
LR          = 1.0      # <-- prueba 0.001 (paso chico) y 1.0 (paso grande)
OCULTAS     = 128      # <-- neuronas de la primera capa: prueba 16 y 256
USAR_RELU   = True     # <-- False: quita las dos ReLU
LIMPIAR_GRADIENTES = True # <-- False: se omite optimizer.zero_grad()
NORMALIZAR  = True     # <-- False: los píxeles entran de 0 a 255
EPOCAS_TT   = 3        # <-- número de épocas

act = nn.ReLU if USAR_RELU else nn.Identity      # Identity = no hace nada
      ↪(sin activación)
modelo_tt = nn.Sequential(
    nn.Flatten(),                                # aplanar
    nn.Linear(784, OCULTAS), act(),               # capa 1
    nn.Linear(OCULTAS, 64), act(),               # capa 2
    nn.Linear(64, 10),                          # salida: 10 dígitos
).to(device)
opt_tt = torch.optim.SGD(modelo_tt.parameters(), lr=LR)
escala = 1.0 if NORMALIZAR else 255.0           # ToTensor ya dividió entre
      ↪255; sin normalizar, lo deshacemos
print("Parámetros de tu modelo:", sum(p.numel() for p in modelo_tt.
      ↪parameters()))

curva_loss, curva_acc = [], []
for epoca in range(1, EPOCAS_TT + 1):
    modelo_tt.train()
```

```

for X, y in train_loader:                                # el mismo ciclo de 5 pasos
    X, y = X.to(device) * escala, y.to(device)
    if LIMPIAR_GRADIENTES:
        opt_tt.zero_grad()                                # reset (salvo que lo
↪apagues)
        perdida = criterion(modelo_tt(X), y)              # forward + pérdida
        perdida.backward()                                # backward
        opt_tt.step()                                    # paso
    modelo_tt.eval(); pt, ac, tot = 0.0, 0, 0              # examen
    with torch.no_grad():
        for X, y in test_loader:
            X, y = X.to(device) * escala, y.to(device)
            s = modelo_tt(X)
            pt += criterion(s, y).item() * X.size(0); ac += (s.argmax(1) == y).
↪sum().item(); tot += X.size(0)
            curva_loss.append(pt / tot); curva_acc.append(ac / tot)
        print(f"Época {epoca}: pérdida de prueba {pt/tot:.3f} | exactitud de prueba
↪{ac/tot*100:.2f}%")

fig, (a1, a2) = plt.subplots(1, 2, figsize=(12, 4))
a1.plot(range(1, EPOCAS + 1), historial["test_loss"], "s--", label="original")
a1.plot(range(1, EPOCAS_TT + 1), curva_loss, "o-", label="tu cambio")
a1.set_title("Pérdida de prueba"); a1.set_xlabel("época"); a1.legend(); a1.
↪grid(alpha=0.3)
if max(curva_loss) > 10: a1.set_yscale("log")              # si se disparó, escala
↪logarítmica para verla
a2.plot(range(1, EPOCAS + 1), historial["test_acc"], "s--", label="original")
a2.plot(range(1, EPOCAS_TT + 1), curva_acc, "o-", label="tu cambio")
a2.set_title("Exactitud de prueba"); a2.set_xlabel("época"); a2.legend(); a2.
↪grid(alpha=0.3)
for a in (a1, a2): a.set_xticks(range(1, max(EPOCAS, EPOCAS_TT) + 1))
plt.tight_layout(); plt.show()

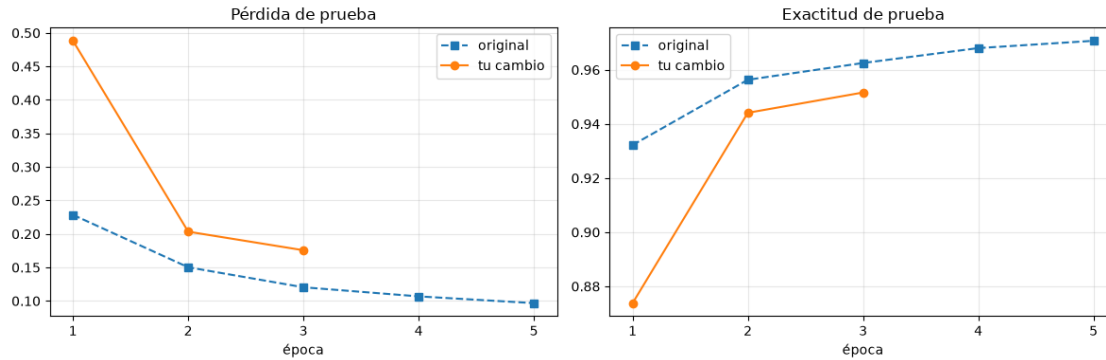
```

Parámetros de tu modelo: 109386

Época 1: pérdida de prueba 0.488 | exactitud de prueba 87.38%

Época 2: pérdida de prueba 0.203 | exactitud de prueba 94.41%

Época 3: pérdida de prueba 0.176 | exactitud de prueba 95.16%



2.14.1 Tu respuesta (escribe aquí)

Pregunta 1: ¿Qué pasó con las curvas al usar un learning rate de 0.001 y uno de 1.0? Explícalo con el ejemplo de echarle sal al arroz.

Cuando el **LR es 0.001** es como si echaras sal granito a granito, pruebas una vez y falta y así vas probando y seguira faltando no se arruinara el arroz pero cuando termina la tercera pasada el arroz sigue desabrido, necesitara muchas pasadas para llegar a su punto. Cuando el **LR es 1.0** le estas echando sal a cucharas, despues de solo 3 el arroz queda rico (en la prueba de arriba 95%) pero nunca en su punto y si te pasas demsiado esta el riesgo de que se quede incomible sin arreglo.

Pregunta 2 (la de la clase): El MLP aprendió igual de bien con los píxeles revueltos. ¿Qué información importante ignora un MLP al aplanar la imagen, y cómo la aprovechará la CNN de la próxima clase?

No aprovechara la forma del dígito, porque la imagen se convierte en una fila de numeros y ya no una matriz, la CNN podria usar este aplanamiento cuando ya hizo sus calculos o uso de sus filtros con pixeles vecinos, al final ya aplanaria el rasgo ya conocido o detectado.